SIMATS ENGINEERING COLLEGE

C PROGRAMMING — CO1 ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program
for a Rural Electricity Co-operative

Tasks 1–4 (including the Day-4 Design Challenge) | Total: 50 marks | Individual, 1 week

Name	AKSHAYA R S
Register Number	1924260209
Course Code	CSA0209B
GitHub Repository Link	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING-1924260209

Contents

- Task 1 — Implement and Test the Billing Program (4 Marks)
- Task 2 — Compare Expectations with Execution (4 Marks)
- Task 3 — Recommend and Defend the Final Design (4 Marks)
- Task 4 — Re-evaluate and Adapt the Design: smart meters and time-of-day tariff (8 Marks)
- Execution and GitHub Record

Task 1 – Implement and Test the Billing Program (4 Marks)

Bloom's level: K3 – Apply.

1.1 Design decisions

- **Money** is held as integer paise in a `long long` and printed as rupees and paise with `%lld.%02lld`, so every bill is exact to the paisa.
- **Meter rollover:** units = `(current - previous) & 65535` (arithmetic modulo 65536), so 65400 to 120 gives 256 units, never a negative bill.
- **Rollover or backwards meter?** A current reading below the previous one is legitimate only if the meter rolled over. The two cases cannot be told apart from the numbers alone, so one rule is used: the wrapped consumption must be at most **5000 units** in a billing period (constant `MAX_PLAUSIBLE_UNITS`). 65400 to 120 gives 256 units and is accepted; 500 to 100 would give 65136 units and is rejected. This limit is an assumption that the co-operative should confirm.
- **Validation** uses `fgets` and `strtoll`: letters, trailing letters (12x), negative numbers and numbers above 65535 are each rejected with a specific message and exit status 1.

1.2 Source code (task01_billing.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <errno.h>

/* ---- Tariff and limits (money in paise) ---- */
#define METER_MAX            65535LL   /* 16-bit meter register */
#define HOUSEHOLD_MAX        999999LL
#define MAX_PLAUSIBLE_UNITS  5000U     /* more than this in one period is treated as a fault
      */
#define SLAB1_UNITS          100U
#define SLAB2_UNITS          200U
#define RATE1_PAISE          0LL
#define RATE2_PAISE          225LL
#define RATE3_PAISE          450LL
#define FIXED_CHARGE_PAISE   5000LL

enum read_status { READ_OK, READ_NOT_NUMBER, READ_NEGATIVE, READ_TOO_BIG, READ_EOF };

/* Reads one line and converts it to a number in 0..max. */ enum read_status
read_number(const char *prompt, long long max, long long *out) {     char
buf[64];     char *end;     const char *p;     long long v;

    printf("%s", prompt);     if (fgets(buf,
sizeof buf, stdin) == NULL)         return
READ_EOF;     p = buf;     while
(isspace((unsigned char)*p))         p++;
errno = 0;     v = strtoll(p, &end, 10);
if (end == p)         return READ_NOT_NUMBER;
while (isspace((unsigned char)*end))
end++;     if (*end != '\0')         return
READ_NOT_NUMBER;     if (*p == '-')
return READ_NEGATIVE;     if (errno == ERANGE
|| v > max)
```

```
        return
READ_TOO_BIG;     *out = v;
return READ_OK; }

/* Prints the reason for a failed read; returns 0 so the caller can exit. */ int
report_error(const char *what, enum read_status st, long long max) {     switch (st)
{     case READ_NOT_NUMBER: printf("Error: %s is not a number.\n", what); break;
case READ_NEGATIVE:   printf("Error: %s must not be negative.\n", what); break;
case READ_TOO_BIG:    printf("Error: %s must be 0 to %lld.\n", what, max); break;
default:              printf("Error: end of input.\n"); break;     }     return 0; }

/* Units consumed, allowing a 16-bit rollover. Returns 0 if the pair of
readings is not plausible (backwards meter or unrealistic consumption). */ int
units_used(unsigned int previous, unsigned int current, unsigned int *units) {
unsigned int u = (current - previous) & (unsigned int)METER_MAX;

    if (u > MAX_PLAUSIBLE_UNITS) {         if (current < previous)
printf("Error: current reading %u is less than previous %u and is not a plausible       "
"rollover (would be %u units; limit %u).\n",                    current, previous, u,
MAX_PLAUSIBLE_UNITS);         else             printf("Error: %u units exceeds the plausible
maximum of %u.\n",                    u, MAX_PLAUSIBLE_UNITS);         return 0;     }
*units = u;     return 1; }

/* Total bill in paise, using the fixed slab tests
*/ long long compute_bill(unsigned int units) {
long long energy;

    if (units <= SLAB1_UNITS)         energy = units *
RATE1_PAISE;     else if (units <= SLAB2_UNITS)
energy = (units - SLAB1_UNITS) * RATE2_PAISE;     else
energy = (SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE
+ (units - SLAB2_UNITS) * RATE3_PAISE;     return energy
+ FIXED_CHARGE_PAISE; }

int main(void) {     long long household,
previous, current;     unsigned int units;
long long total;     enum read_status st;

    if ((st = read_number("Enter household number: ", HOUSEHOLD_MAX, &household)) !=
READ_OK)         { report_error("household number", st, HOUSEHOLD_MAX); return 1; }     if
((st = read_number("Enter previous reading: ", METER_MAX, &previous)) != READ_OK)         {
report_error("previous reading", st, METER_MAX); return 1; }     if ((st =
read_number("Enter current reading: ", METER_MAX, &current)) != READ_OK)         {
report_error("current reading", st, METER_MAX); return 1; }     if (!units_used((unsigned
int)previous, (unsigned int)current, &units))         return 1;

    total = compute_bill(units);     printf("\n--
- Electricity Bill ---\n");     printf("Household
Number: %lld\n", household);
```

```
    printf("Units Consumed: %u\n", units);     printf("Total Bill: Rs
%lld.%02lld\n", total / 100, total % 100);     return 0; }
```

Compile: `gcc task01_billing.c -Wall -Wextra -std=c11 -o t1` *— no warnings. Sample run (inputs 101, 1200, 1530; 330 units = 100 × Rs 2.25 + 130 × Rs 4.50 = Rs 810.00, plus Rs 50.00 fixed):*

```
Enter household number: 101
Enter previous reading: 1200
Enter current reading: 1530

--- Electricity Bill ---
Household Number: 101
Units Consumed: 330
Total Bill: Rs 860.00
```

1.3 Expected values calculated by hand

Case	Units	Working	Bill
N1	60	within the first 100 units: fixed charge only	Rs 50.00
N2	150	50.00 + 50 × 2.25 = 50.00 + 112.50	Rs 162.50
N3	350	50.00 + 100 × 2.25 + 150 × 4.50 = 50 + 225 + 675	Rs 950.00
B1	0	fixed charge only	Rs 50.00
B2	100	the first 100 units are free	Rs 50.00
B3	101	50.00 + 1 × 2.25	Rs 52.25
B4	200	50.00 + 100 × 2.25	Rs 275.00
B5	201	275.00 + 1 × 4.50	Rs 279.50
B6	5000	50 + 225 + 4800 × 4.50 = 50 + 225 + 21600	Rs 21875.00
R1	256	(65536 − 65400) + 120 = 256 units; 50 + 225 + 56 × 4.50 = 50 + 225 + 252	Rs 527.00
R2	1	65535 to 0 is one unit	Rs 50.00
R3	100	65535 to 99 = 1 + 99 = 100 units	Rs 50.00
R4	101	65535 to 100 = 1 + 100 = 101 units; 50 + 2.25	Rs 52.25

1.4 Test table: expected and actual results

Each test feeds the three input lines to the program (run_tests1.py) and checks the output text and the exit status (0 for a bill, 1 for an error). N = normal, B = boundary, R = rollover, I = invalid input.

ID	Case	Input (household / previous / current)	Expected	Actual	Result
N1	Normal, slab 1 (60 units)	101 / 1000 / 1060	Total Bill: Rs 50.00	Total Bill: Rs 50.00	**PASS**
N2	Normal, slab 2 (150 units)	101 / 2000 / 2150	Total Bill: Rs 162.50	Total Bill: Rs 162.50	**PASS**
N3	Normal, slab 3 (350 units)	101 / 500 / 850	Total Bill: Rs 950.00	Total Bill: Rs 950.00	**PASS**
B1	Zero units	101 / 700 / 700	Total Bill: Rs 50.00	Total Bill: Rs 50.00	**PASS**
B2	Exactly 100 units	101 / 100 / 200	Total Bill: Rs 50.00	Total Bill: Rs 50.00	**PASS**
B3	101 units	101 / 100 / 201	Total Bill: Rs 52.25	Total Bill: Rs 52.25	**PASS**
B4	Exactly 200 units	101 / 0 / 200	Total Bill: Rs 275.00	Total Bill: Rs 275.00	**PASS**
B5	201 units	101 / 0 / 201	Total Bill: Rs 279.50	Total Bill: Rs 279.50	**PASS**

B6	5000 units (plausibility limit)	101 / 0 / 5000	Total Bill: Rs 21875.00	Total Bill: Rs 21875.00	**PASS**
R1	Rollover 65400 to 120 (256 units)	102 / 65400 / 120	Total Bill: Rs 527.00	Total Bill: Rs 527.00	**PASS**
R2	Rollover 65535 to 0 (1 unit)	102 / 65535 / 0	Total Bill: Rs 50.00	Total Bill: Rs 50.00	**PASS**
ID	**Case**	**Input (household / previous / current)**	**Expected**	**Actual**	**Result**
R3	Rollover 65535 to 99 (100 units)	102 / 65535 / 99	Total Bill: Rs 50.00	Total Bill: Rs 50.00	**PASS**
R4	Rollover 65535 to 100 (101 units)	102 / 65535 / 100	Total Bill: Rs 52.25	Total Bill: Rs 52.25	**PASS**
I1	Letters in previous reading	103 / abc / 100	previous reading is not a number	Enter household number: Enter previous reading: Error: previous reading is not a number.	**PASS**
I2	Digits followed by a letter (12x)	103 / 100 / 12x	current reading is not a number	Enter household number: Enter previous reading: Enter current reading: Error: current reading is not a number.	**PASS**
I3	Negative previous reading	103 / -5 / 100	previous reading must not be negative	Enter household number: Enter previous reading: Error: previous reading must not be negative.	**PASS**
I4	Negative current reading	103 / 100 / -1	current reading must not be negative	Enter household number: Enter previous reading: Enter current reading: Error: current reading must not be negative.	**PASS**
I5	Current < previous, no rollover	103 / 500 / 100	not a plausible rollover	Enter household number: Enter previous reading: Enter current reading: Error: current reading 100 is less than previous 500 and is not a plausible rollover (would be 65136 units; limit 5000).	**PASS**
I6	Reading above 65535	103 / 100 / 70000	current reading must be 0 to 65535	Enter household number: Enter previous reading: Enter current reading: Error: current reading must be 0 to 65535.	**PASS**
I7	Letters in household number	A12 / 100 / 200	household number is not a number	Enter household number: Error: household number is not a number.	**PASS**
I8	5001 units (above plausibility limit)	103 / 0 / 5001	exceeds the plausible maximum	Enter household number: Enter previous reading: Enter current reading: Error: 5001 units exceeds the plausible maximum of 5000.	**PASS**
I9	Empty input	(none) / (none) / (none)	end of input	Enter household number: Error: end of input.	**PASS**

Result: 22 of 22 tests passed. The tests cover all three slabs, both slab boundaries (100 and 200) with the units either side, zero units, four rollover cases, letters, a negative reading, a backwards meter, an out-of-range reading and an empty input.

1.5 Five bills whose total was calculated by hand

Household	Previous	Current	Expected (by hand)	Actual	Result
101	1000	1080	Rs 50.00	Rs 50.00	**PASS**
102	200	320	Rs 95.00	Rs 95.00	**PASS**
103	300	475	Rs 218.75	Rs 218.75	**PASS**
104	0	260	Rs 545.00	Rs 545.00	**PASS**
105	1200	1530	Rs 860.00	Rs 860.00	**PASS**
Total			**Rs 1768.75**	**Rs 1768.75**	**PASS**

Hand working: 80 units = Rs 50.00; 120 units = 50 + 20 × 2.25 = Rs 95.00; 175 units = 50 + 75 × 2.25 = Rs 218.75; 260 units = 50 + 225 + 60 × 4.50 = Rs 545.00; 330 units = 50 + 225 + 130 × 4.50 = Rs 860.00. Sum: 50.00 + 95.00 + 218.75 + 545.00 + 860.00 = **Rs 1,768.75**, the same as the program's total.

1.6 Alternative: slab limits and rates in two arrays, bill computed by a loop

The limits and rates become data (the last limit is the largest unsigned value), and one loop handles any number of slabs:

```
/* Slabs as data: upper limit (units) and rate (paise per unit) of each slab */
static const unsigned int slab_limit[NSLABS] = { 100U, 200U, UINT_MAX }; static
const long long    slab_rate[NSLABS]  = { 0LL, 225LL, 450LL };
```

```
long long bill_loop(unsigned int units)
{     long long total =
FIXED_CHARGE_PAISE;     unsigned int
lower = 0;

    for (int i = 0; i < NSLABS; i++) {         if (units >
slab_limit[i]) {             total += (long long)(slab_limit[i] -
lower) * slab_rate[i];             lower = slab_limit[i];         }
else {             total += (long long)(units - lower) * slab_rate[i];
break;         }     }     return total;
}
```

Correctness. *The loop version and the if-version were compared for every possible reading difference, 0 to 65535 (diff_slabs.c):*

```
Compared bill_if and bill_loop for units 0..65535: 0 mismatches
units 150: if = 16250, loop = 16250 paise units 350: if =
95000, loop = 95000 paise
```

Measured time per bill *(bench_slabs.c, -O2; the bill is always in the last slab, which is the worst case for the loop):*

```
loop over s =    3 slabs:    4.88 ns per bill   (arrays: 36 bytes)
loop over s =   10 slabs:    8.22 ns per bill   (arrays: 120 bytes)
loop over s =  100 slabs:   77.20 ns per bill   (arrays: 1200 bytes)
loop over s = 1000 slabs:  767.67 ns per bill   (arrays: 12000 bytes)
fixed if-chain (3 slabs):    0.54 ns per bill   (no arrays)
```

Time. The if-version does a fixed number of tests, O(1), here 0.5 ns per bill. The loop does up to s iterations, O(s): 4.9 ns for 3 slabs, 8.2 ns for 10, 77 ns for 100 and 768 ns for 1000 slabs, about 0.77 ns more for each extra slab. For this tariff (s = 3) the difference is about 4 ns per bill: 16 microseconds for the village's 4,000 bills and 3.5 ms for the district's 800,000. **Space.** The loop needs two arrays, (4 + 8) × s bytes: 36 bytes for 3 slabs, O(s); the if-version stores no arrays. **Ease of change.** The array version is easier to change: a new rate, a moved limit or a fourth slab is a change to the data only, and the same loop is used for every slab, so the tariff cannot be applied "only partly". In the if-version the numbers appear inside the formula of each branch and a new slab needs new code and new tests. The cost of the loop is negligible at this size, so the array version is the better choice if the tariff is expected to change; the if-version is the simplest and fastest for a fixed three-slab tariff.

Evidence for Task 1: working source code (1.2) and a test table with expected and actual results for every case (1.3–1.5).

Task 2 – Compare Expectations with Execution (4 Marks)

Bloom's level: K4 – Analyse. All experiments use the same deterministic generator of consumption (0–399 units per household), so every run can be repeated.

2.1 Sizes and ranges compared with the C standard

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void) {     printf("CHAR_BIT = %d\n", CHAR_BIT);     printf("%-14s %6s %8s
%s\n", "type", "bytes", "bits", "range on this system");     printf("%-14s %6zu %8zu  %d
to %d\n", "short", sizeof(short), sizeof(short) * CHAR_BIT,       SHRT_MIN, SHRT_MAX);
    printf("%-14s %6zu %8zu  %d to %d\n", "int", sizeof(int), sizeof(int) *
CHAR_BIT,       INT_MIN, INT_MAX);     printf("%-14s %6zu %8zu  0 to %u\n",
"unsigned int", sizeof(unsigned int),       sizeof(unsigned int) * CHAR_BIT,
UINT_MAX);
    printf("%-14s %6zu %8zu  %ld to %ld\n", "long", sizeof(long), sizeof(long) * CHAR_BIT,
LONG_MIN, LONG_MAX);
    printf("%-14s %6zu %8zu  %lld to %lld\n", "long long", sizeof(long long), sizeof(long
long) * CHAR_BIT, LLONG_MIN, LLONG_MAX);     printf("%-14s %6zu %8zu  FLT_DIG = %d,
FLT_EPSILON = %g, FLT_MAX = %g\n", "float",       sizeof(float), sizeof(float) * CHAR_BIT,
FLT_DIG, (double)FLT_EPSILON, (double)FLT_MAX);     printf("%-14s %6zu %8zu  DBL_DIG = %d,
DBL_EPSILON = %g, DBL_MAX = %g\n", "double",       sizeof(double), sizeof(double) * CHAR_BIT,
DBL_DIG, DBL_EPSILON, DBL_MAX);     return 0;
}
```

Compile: `gcc task02_sizes.c -Wall -Wextra -std=c11 -o t2_sizes` *— no warnings. Output:*

```
CHAR_BIT = 8 type            bytes     bits   range on this system short
2       16  -32768 to 32767 int                 4       32  -2147483648
to 2147483647 unsigned int          4         32   0 to 4294967295 long
8      64  -9223372036854775808 to 9223372036854775807 long long         8
64  -9223372036854775808 to 9223372036854775807
float               4       32  FLT_DIG = 6, FLT_EPSILON = 1.19209e-07, FLT_MAX =
3.40282e+38 double              8       64  DBL_DIG = 15, DBL_EPSILON = 2.22045e-16,
DBL_MAX =
      1.79769e+308
```

Type	Minimum guaranteed by the C standard	Observed here	Difference?
char	CHAR_BIT at least 8	8 bits	No
short	at least 16 bits (±32,767)	2 bytes, 16 bits	No: exactly the minimum
int	at least 16 bits (±32,767)	4 bytes, 32 bits (±2.1 × 10^9)	Yes: twice the minimum
long	at least 32 bits (±2,147,483,647)	8 bytes, 64 bits	Yes: twice the minimum
long long	at least 64 bits	8 bytes, 64 bits	No: exactly the minimum
float	at least 6 decimal digits (FLT_DIG ≥ 6)	FLT_DIG = 6, epsilon 1.19 × 10^-7	No: exactly the minimum
double	at least 10 decimal digits (DBL_DIG ≥ 10)	DBL_DIG = 15, epsilon 2.22 × 10^-16	Yes: much better

Reasons for the differences. The standard fixes only minimum sizes, and each platform chooses its own. This is 64-bit Linux, which uses the LP64 model: int is 32 bits and long is 64 bits. On 32-bit systems and on

64-bit Windows (LLP64) long is only 32 bits, so a program that assumes `long` is 64-bit would overflow there; `long long` is the type guaranteed to be at least 64 bits everywhere. float and double are the IEEE 754 formats (24 and 53 bits of precision), which is why FLT_DIG is 6 and DBL_DIG is 15. The 32-bit build could not be run to confirm (gcc -m32 failed because the 32-bit C library is not installed), so the statement about 4-byte long is based on the standard and the sizes above.

2.2 Float and paise versions of the district total

```
#include <stdio.h>

/* Deterministic pseudo-random consumption 0..399 units per household
*/ static unsigned int seed = 12345U; static unsigned int
next_units(void) {     seed = seed * 1103515245U + 12345U;     return
(seed >> 16) % 400U; }

/* Tariff A: Rs 2.25 and Rs 4.50 (exact in binary); Tariff B (revised): Rs 2.30 and Rs
4.60       (not exact) */ static long long bill_paise(unsigned int u, long long r2, long
long r3) {     long long e = 0;     if (u > 200U)      e = 100LL * r2 + (u - 200U) * r3;
else if (u > 100U) e = (u - 100U) * r2;     return e + 5000LL; } static float
bill_float(unsigned int u, float r2, float r3) {     float e = 0.0f;     if (u > 200U)
e = 100.0f * r2 + (float)(u - 200U) * r3;     else if (u > 100U) e = (float)(u - 100U) *
r2;     return e + 50.0f; } static double bill_double(unsigned int u, double r2, double
r3) {     double e = 0.0;     if (u > 200U)      e = 100.0 * r2 + (double)(u - 200U) * r3;
else if (u > 100U) e = (double)(u - 100U) * r2;     return e + 50.0; }

static void run(const char *name, long long r2, long long r3, float f2, float f3, double
d2,       double d3) {     const long ns[] = { 4000L, 60000L, 800000L };     printf("%s\n",
name);     printf("%8s %20s %20s %10s %20s %8s\n", "bills", "exact paise (long long)",
"float total       (Rs)", "float err", "double total (Rs)", "dbl err");     for (int k = 0;
k < 3; k++) {         long long exact = 0;         float fsum = 0.0f;         double dsum =
0.0;         seed = 12345U;         for (long i = 0; i < ns[k]; i++) {             unsigned
int u = next_units();             exact += bill_paise(u, r2, r3);             fsum  +=
bill_float(u, f2, f3);             dsum  += bill_double(u, d2, d3);         }         long
long fp = (long long)((double)fsum * 100.0 + 0.5);         long long dp = (long long)(dsum
* 100.0 + 0.5);         printf("%8ld %17lld.%02lld %17lld.%02lld %7lld p %17lld.%02lld
%5lld p\n",                ns[k], exact / 100, exact % 100, fp / 100, fp % 100, fp - exact,
dp / 100, dp % 100, dp - exact);     } }

int main(void) {     run("Tariff A: Rs 2.25 / Rs 4.50", 225, 450, 2.25f,
4.5f, 2.25, 4.5);
    run("Tariff B (revised): Rs 2.30 / Rs 4.60", 230, 460, 2.30f, 4.60f, 2.30, 4.60);

    /* Integer accumulators at district scale (Tariff A) */     const long ns[] = { 4000L,
60000L, 800000L };     printf("\nInteger accumulators (total in paise):\n");     printf("%8s
%18s %14s %26s %16s\n", "bills", "true total", "int (32-bit)", "int overflow       at bill
no.", "long long");     for (int k = 0; k < 3; k++) {         long long exact = 0, ll = 0;
int ti = 0, first = 0;         seed = 12345U;         for (long i = 0; i < ns[k]; i++) {
```

```
long long b = bill_paise(next_units(), 225, 450);             exact += b;             ll +=
b;             if (first == 0 && __builtin_add_overflow(ti, (int)b, &ti))
first = (int)i + 1;             else if (first != 0)                 ti = (int)(unsigned
int)((unsigned int)ti + (unsigned int)b);         }         printf("%8ld %18lld %14d %26d
%16lld\n", ns[k], exact, ti, first, ll);     }     printf("sizeof(long) here = %zu bytes, so
a long accumulator %s at 800,000 bills\n",            sizeof(long), sizeof(long) >= 8 ?
"does not overflow" : "overflows");     return 0; }
```

Compile: `gcc task02_totals.c -Wall -Wextra -std=c11 -O0 -o t2_totals` *— no warnings. Each bill is calculated and added to three totals: integer paise (exact), float rupees and double rupees. Tariff A is the present tariff; tariff B is a revision to Rs 2.30 and Rs 4.60, whose rates cannot be stored exactly in binary:*

```
Tariff A: Rs 2.25 / Rs 4.50    bills exact paise (long long)     float total (Rs)  float
err    double total (Rs)  dbl err     4000           1674440.75          1674440.75
0 p          1674440.75     0 p
   60000          24850411.50          24850330.00    -8150 p          24850411.50     0 p
800000         331321357.75         331607200.00 28584225 p         331321357.75     0 p
Tariff B (revised): Rs 2.30 / Rs 4.60    bills exact paise (long long)     float total (Rs)
float err    double total (Rs)  dbl err     4000           1707206.10           1707206.13
3 p          1707206.10     0 p
   60000          25335976.20          25335892.00    -8420 p          25335976.20     0 p
800000         337795165.70         338091520.00 29635430 p         337795165.70     0 p

Integer accumulators (total in paise):
   bills         true total   int (32-bit)   int overflow at bill no.        long long
4000          167444075      167444075                         0        167444075
   60000         2485041150    -1809926146                     51810       2485041150
800000        33132135775    -1227602593                     51810      33132135775
sizeof(long) here = 8 bytes, so a long accumulator does not overflow at 800,000 bills
```

Bills	Exact total (paise version)	Float total	Float minus exact	Double minus exact
Tariff A 4,000	Rs 1,674,440.75	Rs 1,674,440.75	0	0
60,000	Rs 24,850,411.50	Rs 24,850,330.00	−Rs 81.50	0
800,000	Rs 331,321,357.75	Rs 331,607,200.00	+Rs 285,842.25	0
Tariff B 4,000	Rs 1,707,206.10	Rs 1,707,206.13	+3 paise	0
60,000	Rs 25,335,976.20	Rs 25,335,892.00	−Rs 84.20	0
800,000	Rs 337,795,165.70	Rs 338,091,520.00	+Rs 296,354.30	0

Expected versus observed. The float and paise versions do not agree at taluk and district scale. Float agrees exactly with the paise total at 4,000 bills for tariff A (all values used are exact in binary), but is Rs 81.50 wrong at 60,000 bills and Rs 285,842.25 wrong at 800,000. **Reasons:** a float has 24 bits of precision, about 7 digits. A total of Rs 331 million needs 10 digits, and at that size neighbouring floats are 32 apart, so every addition of a bill of about Rs 414 is rounded to a multiple of 32 and the rounding errors accumulate (the final float total, 331,607,200, is exactly divisible by 32). With tariff B an error also appears at 4,000 bills because 2.30 and 4.60 are not exact in binary. A double (53 bits) kept the correct total to the paisa in all six cases.

2.3 Do int, long and long long totals overflow at district scale?

The integer totals (in paise) from the same run are above. The largest int is 2,147,483,647 paise = Rs 21,474,836.47. The average bill is 33,132,135,775 ÷ 800,000 ≈ 41,415 paise, so the expected overflow point is 2,147,483,647 ÷ 41,415 ≈ 51,850 bills. The observed first overflow was at bill number 51,810 (the first bills happened to be a little above the average).

Bills	True total (paise)	int (4 bytes)	long (8 bytes here)	long long

4,000	167,444,075	167,444,075 (correct)	correct	correct
60,000	2,485,041,150	−1,809,926,146 (overflow at bill 51,810)	correct	correct
800,000	33,132,135,775	−1,227,602,593 (overflow at bill 51,810)	correct	correct

A taluk total already overflows an int, and the district total printed as a negative amount. The long total is correct only because long is 8 bytes on this system; where long is 4 bytes it gives the same wrong values as int. `long long` is correct on every system.

2.4 Alternative: accumulate the district total in a double

```
#include <stdio.h>
#include <time.h>
#include <limits.h>

int main(void) {     /* 1. Where does a double stop representing whole
paise exactly? */     double two53 = 9007199254740992.0;            /*
2^53 */     double d = two53 - 3.0;     printf("1) double holding whole
paise near 2^53 = 9007199254740992\n");     for (int i = 0; i < 6; i++) {
d += 1.0;         printf("   after adding 1 paise: %.0f\n", d);     }

    /* 2. long long: no precision loss, but overflow at 2^63 - 1 */     long long x =
LLONG_MAX - 2;     printf("2) long long near the limit %lld\n", LLONG_MAX);     for
(int i = 0; i < 4; i++) {         long long y;         if (__builtin_add_overflow(x,
1, &y))             { printf("   adding 1 to %lld overflows (wraps to %lld)\n", x, y);
break; }         x = y;         printf("   after adding 1 paise: %lld\n", x);     }

    /* 3. Rupee fractions kept in a double: 800,000 bills of Rs 12.34 */     double
rs = 0.0;     long long paise = 0;     for (long i = 0; i < 800000L; i++) { rs +=
12.34; paise += 1234; }     printf("3) 800,000 bills of Rs 12.34: double rupees =
%.6f, exact = %lld.%02lld,       double*100 - exact = %.6f paise\n",            rs,
paise / 100, paise % 100, rs * 100.0 - (double)paise);

    /* 4. Whole paise kept in a double at district scale */     double dp = 0.0;     for
(long i = 0; i < 800000L; i++) dp += 41415.0;      /* a bill of Rs 414.15 in paise */
printf("4) 800,000 bills of 41415 paise held as double: %.0f (exact %lld)\n", dp,
800000LL * 41415LL);

    /* 5. Time per addition and space per variable */
const long long N = 200000000LL;
    double ds = 0.0;     long long ls = 0;     clock_t t0 = clock();     for (long long i =
0; i < N; i++) ds += 1234.0;     clock_t t1 = clock();     for (long long i = 0; i < N; i++)
ls += 1234;     clock_t t2 = clock();     printf("5) %lld additions: double %.3f s (%.2f ns
each), long long %.3f s (%.2f ns each);       results %.0f and %lld\n",            N,
(double)(t1 - t0) / CLOCKS_PER_SEC, (double)(t1 - t0) * 1e9 / CLOCKS_PER_SEC /
      N,          (double)(t2 - t1) / CLOCKS_PER_SEC, (double)(t2 - t1) * 1e9 / CLOCKS_PER_SEC
/ N,          ds, ls);       printf("    sizeof(double) = %zu, sizeof(long long) = %zu\n",
sizeof(double), sizeof(long       long));     return 0; }
```

Compile: `gcc task02_double.c -Wall -Wextra -std=c11 -o t2_double` *— no warnings. Output:*

```
1) double holding whole paise near 2^53 = 9007199254740992    after adding 1 paise:
9007199254740990    after adding 1 paise: 9007199254740991    after adding 1 paise:
9007199254740992    after adding 1 paise: 9007199254740992    after adding 1 paise:
9007199254740992    after adding 1 paise: 9007199254740992 2) long long near the limit
9223372036854775807    after adding 1 paise: 9223372036854775806    after adding 1 paise:
9223372036854775807    adding 1 to 9223372036854775807 overflows (wraps to -
9223372036854775808) 3) 800,000 bills of Rs 12.34: double rupees = 9871999.999912, exact
= 9872000.00, double*100
      - exact = -0.008767 paise
4) 800,000 bills of 41415 paise held as double: 33132000000 (exact 33132000000) 5)
200000000 additions: double 0.595 s (2.98 ns each), long long 0.263 s (1.31 ns each);
results 246800000000 and 246800000000    sizeof(double) = 8, sizeof(long long) = 8
```

	long long	double
How accuracy is lost	**Overflow**: exact up to 9,223,372,036,854,775,807, then wraps to a negative number (test 2).	**Precision loss**: whole numbers are exact only up to 2^53 = 9,007,199,254,740,992; above that, adding 1 changes nothing (test 1). No error is reported.
When it fails (bill of Rs 414.15)	after about 2.2 × 10^14 bills	after about 2.2 × 10^11 bills
At 800,000 bills	exact	exact if whole paise are added (test 4: 33,132,000,000). If rupee fractions are added (800,000 × Rs 12.34) the total is 0.009 paise off, which rounds correctly (test 3).
Time per addition (200 million additions)	1.31 ns (0.263 s)	2.98 ns (0.595 s)
Space per variable	8 bytes	8 bytes

Answer. At 800,000 bills a double accumulator still gives the right total, because the total (3.3 × 10^10 paise) is far below 2^53. It would not be correct to rely on this: the double is exact only while every value is a whole number of paise and the total stays below 2^53, and when it fails it fails silently by dropping small additions, whereas an overflow in a long long can be detected. Both additions cost O(1) time and O(1) space (8 bytes); the double addition was about 2.3 times slower here. The long long is the better accumulator: it is exact for every whole-paise amount up to 9.2 × 10^18.

Evidence for Task 2: sizes against the standard (2.1), float, double and paise totals (2.2), overflow (2.3), double alternative (2.4).

Task 3 – Recommend and Defend the Final Design (4 Marks)

Bloom's level: K5 – Evaluate. Every recommendation below is tied to an experiment in Tasks 1 and 2 or to the experiments of this task.

3.1 Recommended types, constants and preprocessor features

Item	Recommendation	Evidence from my experiments
Money (bills, charges, totals)	`long long` holding whole paise; rupees and paise are produced only when printing	Float totals were wrong by Rs 285,842.25 at 800,000 bills (Task 2.2); int totals overflowed at bill 51,810 (2.3); double was exact but 2.3 times slower and fails silently above 2^53 (2.4).
Meter readings	`unsigned int` (values 0–65535), read as text and range-checked	Rollover tests R1–R4 and invalid-input tests I1–I9 all pass (1.4).
Units consumed	`unsigned int`, computed with `& 65535`	256 units for 65400 → 120; no negative bill in 800,000 test bills (3.3).
Constants	`#define` in one header, each with its unit in the name (RATE2_PAISE), overridable with `-D`	A revision to Rs 2.30 changed one value and gave the hand-checked Rs 165.00 and Rs 955.00 (3.2).
Checks on the constants	`#if ... #error` and `_Static_assert`	An inconsistent tariff is refused at compile time (3.2).
Slab calculation	Fixed if-tests for three slabs; the array and loop form if the number of slabs may change	Identical results for all 65,536 inputs; the loop costs about 4 ns more per bill (1.6).
Rounding	Whole paise only; if a fraction appears, round half up once per bill in integers	Used in Task 4 (tests give Rs 14.40 and Rs 64.40 exactly).

3.2 How a tariff revision is carried out

```
/* ---- Recommended constants: one place, overridable with -D for a revision ---- */
#ifndef SLAB1_UNITS
#define SLAB1_UNITS        100U
#endif
#ifndef SLAB2_UNITS
#define SLAB2_UNITS        200U
#endif
#ifndef RATE1_PAISE
#define RATE1_PAISE        0LL
#endif
#ifndef RATE2_PAISE
#define RATE2_PAISE        225LL
#endif
#ifndef RATE3_PAISE
#define RATE3_PAISE        450LL
#endif
#define FIXED_CHARGE_PAISE 5000LL

/* ---- Compile-time checks of the constants ---- */
#if SLAB1_UNITS >= SLAB2_UNITS
#error "SLAB1_UNITS must be smaller than SLAB2_UNITS"
#endif
#if RATE1_PAISE < 0 || RATE2_PAISE < 0 || RATE3_PAISE < 0
#error "tariff rates must not be negative"
#endif _Static_assert(sizeof(long long) >= 8, "long long must be at least 64
bits");

static long long bill_paise(unsigned int u)
{     long long
e;

    if (u <= SLAB1_UNITS)       e = u * RATE1_PAISE;     else if (u
<= SLAB2_UNITS)  e = (u - SLAB1_UNITS) * RATE2_PAISE;
    else                        e = (SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE + (u
SLAB2_UNITS) * RATE3_PAISE;     return e + FIXED_CHARGE_PAISE; }

/* Independent second implementation (min/max form) used only for checking */ static long
long bill_check(unsigned int u) {     long long a = (u < SLAB1_UNITS) ? u : SLAB1_UNITS;
long long b = (u > SLAB1_UNITS) ? ((u < SLAB2_UNITS ? u : SLAB2_UNITS) - SLAB1_UNITS) : 0;
long long c = (u > SLAB2_UNITS) ? (u - SLAB2_UNITS) : 0;     return FIXED_CHARGE_PAISE + a *
RATE1_PAISE + b * RATE2_PAISE + c * RATE3_PAISE;
}
```

Steps: (1) change the one constant, or pass it with `-D` on the compile line; (2) compile with `-Wall -Wextra`: the `#error` lines refuse a tariff whose slabs overlap or whose rates are negative; (3) run the test table of Task 1 with the new expected values; (4) compare sample bills before and after; (5) commit with a message that states the new tariff and distribute the new program. Example, a revision of the middle-slab rate from 225 to
230 paise, compiled with `gcc -DRATE2_PAISE=230LL task03_district.c` (hand check: 150 units = 50 + 50 × 2.30 = Rs 165.00; 350 units = 50 + 100 × 2.30 + 150 × 4.50 = Rs 955.00):

```
Tariff in use: slabs 100 / 200 units, rates 0 / 230 / 450 paise, fixed 5000 paise
Sample bills: 150 units = Rs 165.00, 350 units = Rs 955.00
```

Compile-time rejection of inconsistent values (`-DSLAB1_UNITS=300U` and `-DRATE3_PAISE=-1LL`):

```
task03_district.c:24:2: error: #error "SLAB1_UNITS must be smaller than SLAB2_UNITS"
task03_district.c:27:2: error: #error "tariff rates must not be negative"
```

3.3 Does the design remain correct at district scale?

task03_district.c bills 4,000, 60,000 and 800,000 generated households. Each bill is checked against a second, independently written formula (min/max form); the total is checked against a histogram method that multiplies the number of households at each consumption by the bill for that consumption; bills are checked for negative values; and the bill is checked to be non-decreasing for every reading from 0 to 65535.

```
Tariff in use: slabs 100 / 200 units, rates 0 / 225 / 450 paise, fixed 5000 paise
Sample bills: 150 units = Rs 162.50, 350 units = Rs 950.00

   bills          total (Rs)  histogram check  mismatch  negative     min bill  ns per bill
4000         1623777.50       1623777.50         0         0        50.00         7.38
   60000        24736914.75      24736914.75         0         0        50.00        15.28
  800000       331069713.25     331069713.25         0         0        50.00        13.44

Bill never decreases as units grow (0..65535): yes
Largest possible single bill (65535 units): 29428250 paise Worst-case district total
(800,000 x largest bill): 23542600000000 paise = 2.4e+13; long long       maximum = 9.2e18
```

- **Correct:** 0 mismatches and 0 negative bills at all three scales; the two totals agree to the paisa; the minimum bill is the Rs 50.00 fixed charge.
- **Bounded:** the largest bill the 16-bit meter can produce is 29,428,250 paise; 800,000 such bills total 2.4 × 10^13 paise, about 390,000 times smaller than the long long limit of 9.2 × 10^18.
- **Scales linearly:** time per bill is about 13 to 15 ns (including generating the readings) from 60,000 to 800,000 bills (4,000 is a little faster, 7 ns, because the data fit in cache). Time is O(n) for n bills and extra space is O(1).

3.4 Limitations of the design

- Three slabs are fixed in the code; a fourth slab needs a code change (the array form of 1.6 removes this).
- A tariff revision needs a recompile, a re-test and a new copy of the program at every office.
- Rollover and a faulty backwards meter cannot be separated by the numbers; the 5000-unit plausibility limit is an assumption, and a meter that rolled over twice, or was replaced, is not detected.
- Only whole units, a single tariff for all customer categories, and no time-of-day rates, refunds or adjustments.
- All tests used generated readings, not real meter data. The 32-bit behaviour of `long` could not be run here.
- Reading the input by hand-typed prompts is not practical for 800,000 households; the program needs file input at that scale (Task 4).

3.5 Alternative: tariff read from a configuration file at start-up

The tariff values move out of the program into a text file of `name number` lines. The program reads, checks and stores them in a structure before billing. Because the checks that `#error` did at compile time must now be done at run time, the loader rejects missing, duplicate, unknown and non-numeric entries, overlapping slabs and negative values.

tariff.cfg:

```
# Grama Jyothi tariff
slab1_units 100
slab2_units 200
rate1_paise 0
rate2_paise 225
rate3_paise 450
fixed_charge_paise 5000
```

```
struct tariff {     long long
slab1, slab2;     long long
rate1, rate2, rate3;     long
long fixed; };

#define NKEYS 6
static const char *keys[NKEYS] = { "slab1_units", "slab2_units", "rate1_paise",
"rate2_paise", "rate3_paise", "fixed_charge_paise" };

/* Reads the tariff file. Returns 1 if the file is complete and consistent, else 0.
*/ int load_tariff(const char *path, struct tariff *t) {     FILE *fp = fopen(path,
"r");     char line[128], key[64], extra;     long long value, vals[NKEYS];     int
seen[NKEYS] = { 0 };     int lineno = 0, k;

    if (fp == NULL) {         printf("Error: cannot open tariff file %s\n", path);
return 0;     }     while (fgets(line, sizeof line, fp) != NULL) {
lineno++;         if (line[0] == '#' || line[0] == '\n')             continue;
/* comment or blank line */         if (sscanf(line, "%63s %lld %c", key, &value,
&extra) != 2) {             printf("Error: %s line %d: expected \"name
number\"\n", path, lineno);             fclose(fp);             return 0;
}         for (k = 0; k < NKEYS && strcmp(key, keys[k]) != 0; k++)
          ;
        if (k == NKEYS) {             printf("Error: %s line %d: unknown name
\"%s\"\n", path, lineno, key);             fclose(fp);             return 0;         }
if (seen[k]) {             printf("Error: %s line %d: \"%s\" given twice\n", path,
lineno, key);             fclose(fp);             return 0;         }         seen[k] =
1;         vals[k] = value;     }     fclose(fp);     for (k = 0; k < NKEYS; k++)
if (!seen[k]) {             printf("Error: %s: \"%s\" is missing\n", path, keys[k]);
return 0;         }     t->slab1 = vals[0]; t->slab2 = vals[1];     t->rate1 = vals[2];
t->rate2 = vals[3]; t->rate3 = vals[4]; t->fixed = vals[5];     if (t->slab1 <= 0 || t-
>slab1 >= t->slab2) {         printf("Error: %s: slab1_units must be positive and
smaller than slab2_units\n",       path);         return 0;     }
    if (t->rate1 < 0 || t->rate2 < 0 || t->rate3 < 0 || t->fixed < 0) {
printf("Error: %s: rates and fixed charge must not be negative\n", path);
return 0;     }     return 1;
}
```

Compile: `gcc task03_config.c -Wall -Wextra -std=c11 -o t3_cfg` *— no warnings. Runs with the good file, a revised file (rate2_paise 230, no recompilation) and five faulty files:*

```
$ ./task03_config tariff.cfg 150 350
Tariff loaded from tariff.cfg in 46.0 microseconds
```

```
  150 units -> Rs 162.50
  350 units -> Rs 950.00

$ ./task03_config tariff_rev.cfg 150 350
Tariff loaded from tariff_rev.cfg in 46.2 microseconds
  150 units -> Rs 165.00
  350 units -> Rs 955.00

$ ./task03_config bad_missing.cfg 150 350
Error: bad_missing.cfg: "rate3_paise" is missing

$ ./task03_config bad_number.cfg 150 350
Error: bad_number.cfg line 5: expected "name number"

$ ./task03_config bad_order.cfg 150 350
Error: bad_order.cfg: slab1_units must be positive and smaller than slab2_units

$ ./task03_config bad_negative.cfg 150 350
Error: bad_negative.cfg: rates and fixed charge must not be negative

$ ./task03_config nofile.cfg 150 350
Error: cannot open tariff file nofile.cfg
```

***Start-up cost** (bench_config.c: reading t values from a file, repeated and averaged):*

```
t =        6 values: load        2.7 microseconds   file        66 bytes   array in memory
48 bytes t =     1000 values: load      118.7 microseconds   file     12890 bytes   array
in memory         8000 bytes t =   100000 values: load    11997.6 microseconds   file
1488890 bytes   array in memory         800000 bytes t =  1000000 values: load   156140.5
microseconds   file  15888890 bytes   array in memory        8000000 bytes
```

Time. Reading t values costs O(t): 2.7 microseconds for the six values of this tariff (46 microseconds for the first, cold read inside the real program), 119 microseconds for 1,000 values, 12 ms for 100,000 and 156 ms for 1,000,000 values; each value is read once, so the cost grows in proportion to t. **Space.** The six values occupy 48 bytes in memory and a 66-byte file; a table of t values needs 8t bytes, O(t). **Against recompiling.** Recompiling and linking this program took 41.8 ms on average, about 900 times the cold read of the six values (46 microseconds), but the recompile is paid once per revision, while the reading is paid at every start (billing runs once a month, so 46 microseconds is negligible). The cost that cannot be measured here, and that is much larger, is re-testing and redistributing the program to every office after each revision; with a file only the small tariff file is replaced. **Trade-off:** the file removes the recompile, but the compile-time checks are lost, so the run-time validation above is essential, and a wrong file that is still valid-looking (a rate of 2250 instead of 225) is accepted.

Recommendation. Keep the types and the integer-paise rule of 3.1 unchanged. Use the single-header constants (3.2) as the design that was built and tested, and move the tariff values into a validated configuration file when changes without recompiling are required, as in Task 4. The measurements show that this costs microseconds per run and no extra types.

Evidence for Task 3: types and constants justified by measurements (3.1), a demonstrated revision (3.2), district-scale validation (3.3), limitations (3.4), and a measured alternative (3.5).

Task 4 – Re-evaluate and Adapt the Design (8 Marks)

Bloom's level: K5 – Evaluate. **Changed requirement:** smart meters send a reading in watt-hours every 15 minutes (35,040 readings per house per year); units used between 6 PM and 10 PM cost 20% more; the tariff values must be changeable without recompiling.

4.1 What changes in the problem

	Original design	Smart-meter design
Readings per house	2 per month	96 per day = 2,880 per month = 35,040 per year (96 × 365)
Village (4,000 houses)	8,000 readings per month	11.52 million per month; 140.16 million per year
Taluk (60,000 houses)	120,000 per month	172.8 million per month; 2.10 billion per year
District (800,000 houses)	1.6 million per month	2.304 billion per month; 28.03 billion per year
Reading	cumulative 16-bit register, rollover	energy in Wh for each 15-minute interval
Price depends on	total units in the month	total in the month and the time of day of every reading
Tariff values	compiled into the program	must be changeable without recompiling
Data size, district month	tiny	about 28.8 GB as text (12.5 bytes per line, measured) or 4.6 GB as 2-byte binary values

4.2 Decision: retain, modify or redesign each part

Part of the original design	Decision	Reason and evidence
Money as integer values, rounded by one stated rule	**RETAIN**	Floating point lost Rs 285,842.25 at district scale (Task 2.2). The 20% surcharge on watt-hours creates fractions of a paisa, so integer arithmetic with one rounding per bill matters more than before.
long long for amounts and totals	**RETAIN**	int overflowed at bill 51,810 (Task 2.3). The finest cost unit is now 1/100,000 paisa; the largest possible bill in that unit (28,800 kWh, an upper bound of 2,880 readings of 10,000 Wh at the top rate with the 20% extra) is 1.6 × 10^12, far below 9.2 × 10^18.
Constants in one header, checked at compile time	**MODIFY**	Tariff values move to a file (the board's requirement; Task 3.5 measured the cost: microseconds). Only type limits stay as constants. The checks that #error did are repeated at run time by the loader.
Units as whole kWh; rollover mask	**MODIFY**	Readings are energy per interval in Wh, not a register, so the rollover calculation is no longer needed in the billing path. Slab limits are held in Wh (kWh × 1000) and a reading can fall across a slab limit (tested with household 9).
Validation of every input	**RETAIN, extended**	Each line is checked: four whole numbers, no extra text, slot 0–95, day 1–31, 0 to 10,000 Wh. A bad line is rejected and counted; the run continues.

Three fixed if-tests per bill	**REDESIGN**	The price of a reading depends on the energy already used in the month and on the time of day. Each reading is priced when it arrives (4.3), so a bill is the sum of 2,880 priced readings.
Prompting for three numbers with fgets	**REDESIGN**	Nobody can type 2,880 readings per house. The program reads a file line by line and bills each household when its readings end.
Single source file and one total function	**REDESIGN**	Pricing is separate (smart.h), so it can be tested and benchmarked alone; tariff loading, line checking and billing are separate functions.

Overall decision: REDESIGN the program, RETAINING the proven core (integer money, long long totals, strict validation, one rounding rule). A modification of the old program would not be enough because the unit of work changes from one bill computed from two numbers to a stream of 2.3 billion readings per month for the district, and the input method, tariff storage and pricing all change.

4.3 Rules adopted for the new tariff (assumptions to be confirmed by the board)

- A reading belongs to hour *slot / 4* (slot 0 = 00:00–00:15). The peak window is 6 PM to 10 PM, i.e. hours 18 to 21, slots 72 to 87; 10:00 PM exactly (slot 88) is off-peak.
- The slabs (first 100 kWh free, next 100 kWh at 225 paise per kWh, above 200 kWh at 450 paise) apply to the month's energy in the order the readings arrive. A reading that crosses a slab limit is split between the two slabs.
- Energy used in the peak window costs 20% more: the rate of the slab it falls in is multiplied by 120%. (peak_percent = 120 in the file.)
- Cost of a reading = Wh × rate (paise per kWh) × percent, an integer in 1/100,000 paise. The month's costs are added exactly; the sum is **rounded half up to whole paise once per bill**, then the fixed charge is added.
- Each reading is the energy used in that interval (not a cumulative value); readings of one household are consecutive lines in time order.

4.4 Tariff file and source code

```
# Tariff file: change a value here, no recompilation needed
slab1_limit_kwh 100 slab2_limit_kwh 200 rate1_paise 0
rate2_paise 225 rate3_paise 450 fixed_charge_paise 5000
peak_start_hour 18 peak_end_hour 22

peak_percent 120
```

smart.h — data structure and pricing (also used by the benchmark):

```
#ifndef SMART_H
#define SMART_H

#define SLOTS_PER_DAY      96        /* 15-minute intervals */
#define MAX_WH_PER_READING 10000LL   /* 40 kW for 15 minutes: larger values are rejected */
#define COST_UNIT_PER_PAISE 100000LL /* 1 Wh x 1 paise/kWh x 100 (percent) = 1/100000 paise */

struct tariff {     long long limit1_wh, limit2_wh;      /* slab limits
in watt-hours */     long long rate1, rate2, rate3;       /* paise per
kWh */     long long fixed;                     /* paise per bill */
int peak_start, peak_end;            /* peak hours [start, end) */
long long peak_percent;              /* 120 = 20% more */ };

/* Cost of one reading of wh watt-hours, given the month's energy so far (cum_wh).    The
reading is split across slab boundaries; the result is in 1/100000 paise. */ static inline
long long price_reading(const struct tariff *t, long long cum_wh, long long wh,       int
peak) {     long long mult = peak ? t->peak_percent : 100;     long long end = cum_wh + wh,
cost = 0, lo, hi;

    lo = cum_wh;                                    /* slab 1: [0, limit1)
*/     hi = end < t->limit1_wh ? end : t->limit1_wh;     if (hi > lo) cost
+= (hi - lo) * t->rate1 * mult;



    lo = cum_wh > t->limit1_wh ? cum_wh : t->limit1_wh;     /* slab 2: [limit1, limit2)
*/     hi = end < t->limit2_wh ? end : t->limit2_wh;     if (hi > lo) cost += (hi - lo) *
t->rate2 * mult;

    lo = cum_wh > t->limit2_wh ? cum_wh : t->limit2_wh;     /* slab 3: [limit2, ...)
*/     hi = end;     if (hi > lo) cost += (hi - lo) * t->rate3 * mult;     return
cost; }

/* Rounding rule: round half up, once per bill */ static inline
long long cost_to_paise(long long cost) {     return (cost +
COST_UNIT_PER_PAISE / 2) / COST_UNIT_PER_PAISE; }

static inline int is_peak(const struct tariff *t, int
slot) {     int hour = slot / 4;     return hour >= t-
>peak_start && hour < t->peak_end; }

#endif /* SMART_H */
```

task04_smart.c — loads and validates the tariff, checks and prices each line, prints one bill per household:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>
#include "smart.h"

#define NKEYS 9 static const char *keys[NKEYS] = { "slab1_limit_kwh",
"slab2_limit_kwh", "rate1_paise",       "rate2_paise",
    "rate3_paise", "fixed_charge_paise", "peak_start_hour", "peak_end_hour", "peak_percent" };

/* Reads the tariff file; returns 1 if complete and consistent.
*/ static int load_tariff(const char *path, struct tariff *t) {
FILE *fp = fopen(path, "r");     char line[128], key[64], extra;
long long v, vals[NKEYS];
    int seen[NKEYS] = { 0 }, lineno = 0, k;

    if (fp == NULL) { printf("Error: cannot open tariff file %s\n", path); return 0; }
while (fgets(line, sizeof line, fp) != NULL) {         lineno++;         if (line[0] == '#'
|| line[0] == '\n') continue;         if (sscanf(line, "%63s %lld %c", key, &v, &extra) !=
2) {             printf("Error: %s line %d: expected \"name number\"\n", path, lineno);
fclose(fp); return 0;         }         for (k = 0; k < NKEYS && strcmp(key, keys[k]) != 0;
k++) ;         if (k == NKEYS) { printf("Error: %s line %d: unknown name \"%s\"\n", path,
lineno,       key); fclose(fp); return 0; }         if (seen[k]) { printf("Error: %s line
%d: \"%s\" given twice\n", path, lineno, key);       fclose(fp); return 0; }
seen[k] = 1; vals[k] = v;     }     fclose(fp);     for (k = 0; k < NKEYS; k++)         if
(!seen[k]) { printf("Error: %s: \"%s\" is missing\n", path, keys[k]); return 0; }     t-
>limit1_wh = vals[0] * 1000; t->limit2_wh = vals[1] * 1000;     t->rate1 = vals[2]; t-
>rate2 = vals[3]; t->rate3 = vals[4]; t->fixed = vals[5];     t->peak_start = (int)vals[6];
t->peak_end = (int)vals[7]; t->peak_percent = vals[8];     if (vals[0] <= 0 || vals[0] >=
vals[1]) { printf("Error: %s: slab1_limit_kwh must be       positive and below
slab2_limit_kwh\n", path); return 0; }     if (t->rate1 < 0 || t->rate2 < 0 || t->rate3 < 0
|| t->fixed < 0) { printf("Error: %s:
      rates and fixed charge must not be negative\n", path); return 0; }     if
(t->peak_start < 0 || t->peak_end > 24 || t->peak_start >= t->peak_end) {
```

```
      printf("Error: %s: peak hours must satisfy 0 <= start < end <= 24\n", path); return 0; }
if (t->peak_percent < 100 || t->peak_percent > 300) { printf("Error: %s: peak_percent      must
be 100 to 300\n", path); return 0; }     return 1; }

/* Parses the next whole number from *p; returns 0 if there is none. */ static int
next_number(char **p, long long *out) {     char *end;     while (isspace((unsigned
char)**p)) (*p)++;     if (**p == '\0') return 0;     errno = 0;     *out = strtoll(*p,
&end, 10);     if (end == *p || errno == ERANGE || (!isspace((unsigned char)*end) &&
*end != '\0'))       return 0;     *p = end;     return 1; }

static struct tariff T; static long long cur_house = -
1, cum_wh, peak_wh, cost; static long
district_paise_count; static long long district_total;
static long houses, accepted, rejected;

static void flush_bill(void) {     long long energy, total;     if (cur_house < 0)
return;     energy = cost_to_paise(cost);     total = energy + T.fixed;
printf("Household %lld: %lld.%03lld kWh (peak %lld.%03lld kWh)  energy Rs %lld.%02lld
fixed Rs %lld.%02lld  total Rs %lld.%02lld\n",            cur_house, cum_wh / 1000,
cum_wh % 1000, peak_wh / 1000, peak_wh % 1000,            energy / 100, energy % 100,
T.fixed / 100, T.fixed % 100, total / 100, total %       100);     district_total +=
total;     district_paise_count++;     houses++;     cum_wh = peak_wh = cost = 0; }

int main(int argc, char
**argv) {     FILE *fp;
char line[256], *p;     long
lineno = 0;

    if (argc != 3) { printf("usage: %s tariff.cfg readings.txt\n", argv[0]); return 2;
}     if (!load_tariff(argv[1], &T)) return 1;     fp = fopen(argv[2], "r");     if (fp
== NULL) { printf("Error: cannot open %s\n", argv[2]); return 1; }

    while (fgets(line, sizeof line, fp) != NULL) {         long long h, day, slot, wh;
const char *why = NULL;         lineno++;         p = line;         if (!next_number(&p, &h)
|| !next_number(&p, &day) || !next_number(&p, &slot) ||       !next_number(&p, &wh))
why = "not four whole numbers";         else {             while (isspace((unsigned char)*p))
p++;             if (*p != '\0')                                 why = "extra text after the
reading";             else if (h <= 0)                                 why = "household number
must be       positive";             else if (day < 1 || day > 31)                why = "day
must be 1 to 31";             else if (slot < 0 || slot >= SLOTS_PER_DAY)  why = "slot must
be 0 to 95";             else if (wh < 0)                                 why = "watt-hours must
not be       negative";             else if (wh > MAX_WH_PER_READING)            why = "watt-
hours above the 10000
```

```
      limit";          }          if (why != NULL) { printf("Line %ld rejected: %s\n",
lineno, why); rejected++;        continue; }          if (h != cur_house) { flush_bill();
cur_house = h; }          {              int peak = is_peak(&T, (int)slot);              cost
+= price_reading(&T, cum_wh, wh, peak);              cum_wh += wh;              if (peak)
peak_wh += wh;          }          accepted++;      }      fclose(fp);      flush_bill();
printf("Summary: %ld household(s) billed, %ld readings accepted, %ld rejected, total Rs
%lld.%02lld\n",            houses, accepted, rejected, district_total / 100, district_total
% 100);      return 0; }
```

Compile: `gcc task04_smart.c -Wall -Wextra -std=c11 -o t4_smart` — no warnings. Run: `./t4_smart tariff4.cfg readings_test.txt`.

4.5 Validation of the new design

The test file (readings_test.txt, 50 lines) was built so that every expected value can be calculated by hand:

Household	Readings	Hand calculation	Expected bill
7	60 kWh off-peak, 50 kWh in the peak, then 100 kWh off-peak (21 readings of 10,000 Wh)	Peak reading starts at 60 kWh: 40 kWh free, 10 kWh in slab 2 at 225 × 1.2 = 270 → 2,700. Last 100 kWh from 110 to 210: 90 × 225 = 20,250 and 10 × 450 = 4,500. Energy 27,450 paise = Rs 274.50, plus Rs 50.00. (Check: without the peak rule the bill would be Rs 320.00; the difference Rs 4.50 is 20% of 10 kWh × Rs 2.25.)	Rs 324.50
8	100 kWh off-peak, then one peak reading of 5,333 Wh (not a whole kWh)	100 kWh free. 5.333 kWh × 270 = 1,439.91 paise, rounded to 1,440 = Rs 14.40; plus Rs 50.00	Rs 64.40
9	95 kWh off-peak, then one peak reading of 10,000 Wh that crosses the 100 kWh limit	5 kWh free, 5 kWh × 270 = 1,350 paise = Rs 13.50; plus Rs 50.00	Rs 63.50
12	6 invalid lines and 1 valid reading of 2,000 Wh	Invalid lines are rejected and not billed; 2 kWh is free; fixed charge only	Rs 50.00

Actual output. The six invalid lines of household 12 are "abc" for the Wh value, −40 Wh, slot 96, slot −1, 20,000 Wh and extra text; they are reported when read, so their messages appear before the bill of household 9, which is printed only when household 12 starts:

```
Household 7: 210.000 kWh (peak 50.000 kWh)  energy Rs 274.50  fixed Rs 50.00  total Rs 324.50
Household 8: 105.333 kWh (peak 5.333 kWh)  energy Rs 14.40  fixed Rs 50.00  total Rs 64.40
Line 44 rejected: not four whole numbers
Line 45 rejected: watt-hours must not be negative
Line 46 rejected: slot must be 0 to 95
Line 47 rejected: slot must be 0 to 95
Line 48 rejected: watt-hours above the 10000 limit
Line 49 rejected: extra text after the reading
Household 9: 105.000 kWh (peak 10.000 kWh)  energy Rs 13.50  fixed Rs 50.00  total Rs 63.50
Household 12: 2.000 kWh (peak 0.000 kWh)  energy Rs 0.00  fixed Rs 50.00  total Rs 50.00
Summary: 4 household(s) billed, 44 readings accepted, 6 rejected, total Rs 502.40
```

All four bills equal the hand-calculated values, the six invalid lines are rejected for the right reasons, and the summary total (324.50 + 64.40 + 63.50 + 50.00 = Rs 502.40) is correct.

Changing the tariff without recompiling. The same compiled program, run with changed files (hand checks: rate2 = 230: household 7 has 10 kWh at 230 × 1.2 = 276 → 2,760, 90 × 230 = 20,700 and 4,500, total 27,960 = Rs 279.60, bill Rs 329.60; household 8: 5.333 × 276 = 1,471.9 → Rs 14.72, bill Rs 64.72; peak 150%: 10 kWh at 337.5 → 3,375, total energy 28,125, bill Rs 331.25):

```
./t4_smart tariff4_rev.cfg readings_test.txt    (rate2_paise 230)
```

```
./t4_smart tariff4_p150.cfg readings_test.txt (peak_percent 150)
```

```
Household 7: 210.000 kWh (peak 50.000 kWh)  energy Rs 279.60  fixed Rs 50.00  total Rs 329.60
Household 8: 105.333 kWh (peak 5.333 kWh)  energy Rs 14.72  fixed Rs 50.00  total Rs 64.72
Line 44 rejected: not four whole numbers


Household 7: 210.000 kWh (peak 50.000 kWh)  energy Rs 281.25  fixed Rs 50.00  total Rs 331.25
```

A faulty file is refused before any bill is printed (peak_end_hour 17 is earlier than peak_start_hour 18):

```
Error: bad4.cfg: peak hours must satisfy 0 <= start < end <= 24
```

4.6 Measurements at village, taluk and district scale

bench_smart.c prices every reading of a generated month (2,880 per household, 0–199 Wh each) with the same pricing function as the program, one reading at a time, storing nothing (-O2):

```
    4000 houses:      11520000 readings, total Rs 2737423.12, 0.03 s, 2.77 ns per reading,
peak memory 1160 KB    60000 houses:     172800000 readings, total Rs 41083390.87, 0.51 s,
2.96 ns per reading,       peak memory 1160 KB   800000 houses:    2304000000 readings,
total Rs 547691580.59, 6.62 s, 2.88 ns per reading,       peak memory 1160 KB
```

Reading and checking the text file is much slower than pricing: a file of 200 households (576,000 readings, 7.2 MB) was processed by `task04_smart` *in 0.083 s:*

```
Summary: 200 household(s) billed, 576000 readings accepted, 0 rejected, total Rs 137065.23
file of 200 households (576,000 readings): 0.083 s = 145 ns per reading; extrapolated to
2,304,000,000 readings: 333 s
```

	Original design	Smart-meter design
Time	O(1) per bill; 800,000 bills about 10 ms	O(R) per bill with R = 2,880 readings (a constant per month), so O(n · R) for n households. Pricing: 0.03 s, 0.51 s and 6.6 s for 4,000, 60,000 and 800,000 households, about 2.9 ns per reading, growing in proportion to the number of readings.
Time with a text file	negligible	about 145 ns per reading: 1.7 s for a village, 25 s for a taluk and about 330 s (5.6 minutes) for the district
Space	O(1)	**O(1) streaming:** peak memory 1,160 KB at every scale. Storing one household's readings would need O(R) = 5.8 KB at 2 bytes each; storing the district's, O(n · R) = 4.6 GB

Conclusions. (1) The pricing rule is cheap: the whole district month takes under 7 seconds, and memory does not grow with the number of households because each household is billed and forgotten. (2) The input format is the bottleneck: as text, the district month is 28.8 GB and takes minutes to parse. For the district the readings should be exchanged as binary records (2 bytes per reading) or processed by feeder or in parallel; text is acceptable for a village or a taluk. (3) Because correctness is exact, the larger data volume does not change the accuracy: the district total is a sum of whole-paise bills in a long long.

4.7 Limitations and risks

- The slab-and-peak rule of 4.3 is an interpretation of the board's statement; a different rule (for example surcharge on the bill, not the rate) gives different bills. The rule is isolated in one function, price_reading, so it can be changed without touching the rest.
- Missing, duplicate or out-of-order intervals are not detected (a household with fewer than 2,880 readings is billed for what arrived). A counter of readings per household would reveal gaps.
- A valid-looking but wrong tariff file is accepted; the file should be reviewed and kept in version control.

- The 10,000 Wh limit per reading (40 kW) is a plausibility assumption. Tests used generated data; real meter data may expose cases not considered.

Evidence for Task 4: the decision with reasons for each part (4.2), the rules adopted (4.3), working code (4.4), hand-checked tests, tariff change without recompiling and invalid input (4.5), and measurements at the three scales (4.6).

Execution and GitHub Record

Repository: **CSA02-<RegisterNumber>-Assignment1**, with one folder for each task containing the source code, input files and output screenshots. Commit each task separately.

Folder	Files	Compile and run
Task01	task01_billing.c, run_tests1.py, diff_slabs.c, bench_slabs.c	`gcc task01_billing.c -Wall -Wextra -std=c11 -o t1 python3 run_tests1.py gcc diff_slabs.c -Wall -Wextra -std=c11 -o diff_slabs gcc bench_slabs.c -Wall -Wextra -std=c11 -O2 -o bench_slabs`
Task02	task02_sizes.c, task02_totals.c, task02_double.c	`gcc task02_sizes.c -Wall -Wextra -std=c11 -o t2_sizes gcc task02_totals.c -Wall -Wextra -std=c11 -O0 -o t2_totals gcc task02_double.c -Wall -Wextra -std=c11 -o t2_double`
Task03	task03_district.c, task03_config.c, bench_config.c, tariff.cfg and the bad_*.cfg files	`gcc task03_district.c -Wall -Wextra -std=c11 -o t3_dist gcc -DRATE2_PAISE=230LL task03_district.c -Wall -Wextra -std=c11 -o t3_rev gcc task03_config.c -Wall -Wextra -std=c11 -o t3_cfg ./t3_cfg tariff.cfg 150 350`
Task04	smart.h, task04_smart.c, bench_smart.c, tariff4.cfg, tariff4_rev.cfg, tariff4_p150.cfg, readings_test.txt	`gcc task04_smart.c -Wall -Wextra -std=c11 -o t4_smart ./t4_smart tariff4.cfg readings_test.txt gcc bench_smart.c -Wall -Wextra -std=c11 -O2 -o bench_smart ./bench_smart 800000`

Compiler: `gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0`. Every program compiled with no warnings under `-Wall -Wextra -std=c11`. Long source and output lines are wrapped when printed in this report (the files themselves are not wrapped). Every number in this report was produced by running the programs; timings are from one run on one machine and vary by a few percent from run to run.

README.md must contain: name, register number, compiler and version, the exact compile command and the steps to run each program, and a short note of the result of each task (for example: “Task 1: 22 of 22 tests and the five-bill total Rs 1,768.75 pass”). **Commits:** at least one for each task, for example “Task01: billing program and test table”, “Task02: type experiments”, “Task03: final design and config alternative”, “Task04: smart-meter redesign”. Paste the repository link on the cover page and share the repository with the faculty.